TPOS — Architecture Decision Summary

Trailer Portal Operating System · one-page front-sheet for the full Product Specification · pairs with **schema.sql** + **openapi.yaml**

THE SHAPE OF THE SYSTEM

A multi-tenant B2B SaaS: independent trailer-resale dealerships sign up and run their whole operation — inventory, leads, deals, quotes, reconditioning, titles, invoicing, delivery, and a public storefront — on one shared platform. **One trailer record is viewed through many role lenses** (inventory item → deal → work order → invoice → title → delivery → public listing). Sold three ways: **monthly hosted**, **annual license**, or **one-time perpetual purchase**.

HEADLINE DECISIONS & WHY

Database	**PostgreSQL**, single primary + read replicas. Relational integrity, mature tooling, RLS for tenant isolation. No need for NoSQL — the domain is relational.
Tenant isolation	**Shared schema + Row-Level Security** (`tenant_id` column, `tenant_isolation` policy). RLS overhead is ~1–5% *when* indexes lead with `tenant_id`; the missing index — not RLS — is the 100× penalty.
App architecture	**Modular monolith** behind one API, split into modules (sales, inventory, service, billing, storefront). Microservices add ops cost we don't need at this scale; the module seams let us extract later if a service earns it.
Connection pooling	**PgBouncer in transaction mode**; set the tenant with `SET LOCAL app.tenant_id` *per transaction*. Session-level SET leaks across pooled clients — `SET LOCAL` is the density lever (~3–5 conns/core).
Reads / reporting	**Materialized read models** (KPI views) refreshed on a schedule; CQRS-lite. Dashboards stay fast without taxing the write path.
Storefront	**Edge-cached static-first** public pages reading a published-inventory view. Buyer traffic never touches the operational DB hot path.
Scaling next	**Hash-partition** the largest tables by `tenant_id` into a fixed partition count; add replicas. Single-node Postgres + partitioning + replicas goes very far.

REFERENCE STACK (ILLUSTRATIVE)

Datastore	PostgreSQL 16 · RLS · PgBouncer
API	One service — Django/DRF *or* NestJS
Auth	Session cookie (staff) + bearer (API/portal)
Storefront	Static/SSR, CDN-cached
Storage	Object store for photos/docs
Async	Queue for email, soft-pulls, exports

The spec is implementation-agnostic — language & framework are the rebuilder's choice. These are the load-bearing *constraints*, not mandates.

ROADS NOT TAKEN

Citus / sharding	Deferred — premature at launch scale; revisit past single-node limits.
DB-per-tenant	Migration & connection sprawl; RLS gives isolation more cheaply.
Microservices	Ops & distributed-txn cost unjustified now; monolith first.
Rust/Go for speed	Runtime rarely the bottleneck for DB-bound CRUD; DB & pooling dominate.

Full reasoning & citations: **Appendix D** of the Product Specification.

IF PERFORMANCE WERE THE PRIMARY AXIS

GOAL	FIRST LEVER	THEN
p99 latency	Index `tenant_id`-leading; materialized read models	Replica reads; partition hot tables
Throughput	PgBouncer txn-pooling; batch writes	Horizontal API; read replicas
Tenants/server (density)	Shared-schema RLS; `SET LOCAL` pooling	Hash-partition; tiered storage

Figures are order-of-magnitude — **benchmark your own queries**; some cited sources are vendor/single-practitioner framing.